

The Foundation - System Mission, Ubuntu, Terminal, and GitHub

DR. TNOURJI, ABDELLAH

Research Scientist

The UM6P Vanguard Center

Autonomous Drone Engineering Internship

Mohammed VI Polytechnic University, UM6P , September 8, 2026



University
Mohammed VI
Polytechnic

THE UM6P
VANGUARD
CENTER

#Empowering Minds.

1 Mission and Foundations

1 Mission and Foundations

First, Observe the Destination



The Internship Mission

Build a reproducible autonomous-drone inspection system in simulation

Engineering Need

Inspection tasks may involve large, repetitive, difficult-to-access, or potentially hazardous areas. Aerial robots can help collect visual evidence efficiently and consistently.

The Internship Mission

Build a reproducible autonomous-drone inspection system in simulation

Engineering Need

Inspection tasks may involve large, repetitive, difficult-to-access, or potentially hazardous areas. Aerial robots can help collect visual evidence efficiently and consistently.

Internship Mission

Configure, program, simulate, test, and document a quadcopter that:

- ▶ follows a planned inspection mission;
- ▶ observes a simulated scene with a camera;
- ▶ detects selected objects;
- ▶ records useful mission evidence.

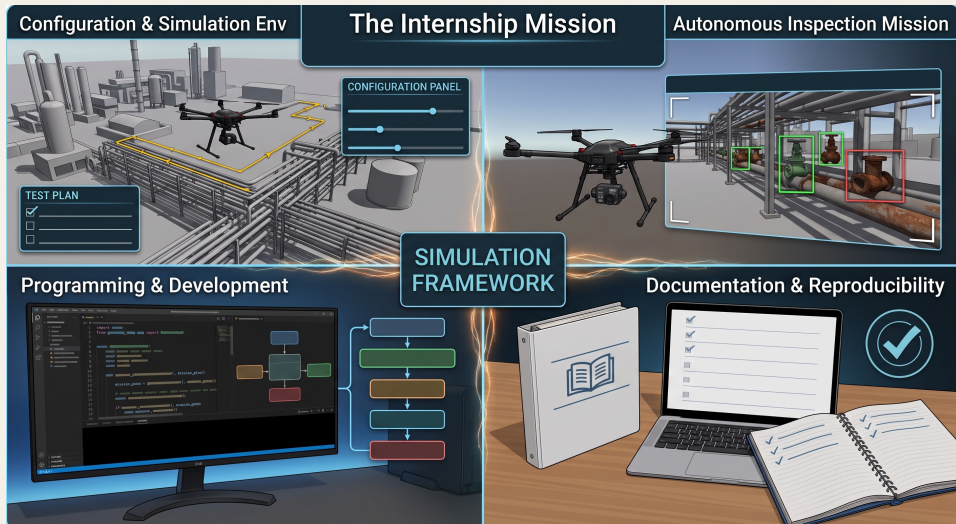
Observe the complete demonstration before studying its components.

Gazebo world → PX4-controlled flight → ROS 2 data exchange → Camera perception → Mission evidence

Important Boundary

A successful simulation is evidence of software integration, not proof that a real aircraft is ready or safe to fly.

First, Observe the Destination



The Engineering Challenge

One visible mission depends on several connected technical layers

Why Is the Project Technically Interesting?

A drone inspection mission is a **system-of-systems problem**. Motion, sensing, communication, software, perception, testing, and documentation must work together.

Operating environment → **Simulation** → **Flight control** → **Communication** → **Perception** →
Evidence

The Engineering Challenge

One visible mission depends on several connected technical layers

Why Is the Project Technically Interesting?

A drone inspection mission is a **system-of-systems problem**. Motion, sensing, communication, software, perception, testing, and documentation must work together.

Operating environment → **Simulation** → **Flight control** → **Communication** → **Perception** → **Evidence**

A Chain Is Verified Link by Link

A detector cannot process a missing camera stream. A mission node cannot guide the vehicle through an incorrect interface. A working local setup cannot be reproduced without controlled dependencies and documentation.

One Project, Several Foundations

A complete robotic system is built layer by layer

The drone demonstration is the visible result.

Behind it, several technical foundations must work together.

One Project, Several Foundations

A complete robotic system is built layer by layer

The drone demonstration is the visible result.

Behind it, several technical foundations must work together.

Prepare

Configure the computer and create a reproducible project environment.

Build

Program, simulate, connect, and test the robotic components.

Deliver

Document the work, preserve the results, and share a working project.

One Project, Several Foundations

A complete robotic system is built layer by layer

The drone demonstration is the visible result.

Behind it, several technical foundations must work together.

Prepare

Configure the computer and create a reproducible project environment.

Build

Program, simulate, connect, and test the robotic components.

Deliver

Document the work, preserve the results, and share a working project.

Think of the Project as a Team

Ubuntu provides the workplace, Bash executes instructions, Git preserves progress, Python implements logic, ROS 2 connects components, Gazebo provides the simulated world, and AI interprets camera data.

You are not expected to master everything on Day 1.

You will learn each foundation when the project needs it.

Technology Ecosystem I

Preparing the environment and preserving your work

Ubuntu: The Workplace

Ubuntu is the Linux distribution used to run the project. It provides the files, processes, packages, and services required by the robotics tools.

Git: The Project Timeline

Git records changes on the computer. It allows students to create checkpoints, compare versions, and recover a previous working state.

Terminal and Bash: The Control Desk

The **terminal** is the window where commands are entered. **Bash** is the shell that reads and executes those commands, variables, and scripts.

GitHub: The Shared Project Home

GitHub hosts the Git repository online. It supports collaboration, documentation, issue tracking, review, and delivery of the internship project.

How We Will Study Each Foundation

A repeated reasoning pattern for every tutorial

1. **Why?** Which project problem does this foundation solve?
2. **What?** What does it mean at an intuitive and technically correct level?
3. **How does it fit?** Where is it located in the complete pipeline?
4. **Project connection?** Which file, process, command, node, topic, or result uses it?
5. **Verification?** What observable output confirms correct behavior?
6. **Next step?** Which later tutorial develops the idea in greater depth?

Example

Before studying ROS 2 details, students first recognize its system role: it carries typed information between software components and makes their interfaces observable and testable.

Professional Working Method

Progress is created through disciplined observation

Before Execution

- ▶ identify the current directory and environment;
- ▶ predict the expected output;
- ▶ understand which component will be affected;
- ▶ change one factor at a time.

After Execution

- ▶ read complete outputs and error messages;
- ▶ compare observation with expectation;
- ▶ preserve commands, logs, and screenshots;
- ▶ document results and limitations honestly.

Engineering Habit

A desired trajectory, a clean visualization, or a single successful launch is not sufficient evidence. Each important claim must be supported by an observable test.

Internship Deliverables

What students are expected to build and submit

Working Engineering Assets

- ▶ structured Git repository and clear history;
- ▶ reproducible environment and configuration;
- ▶ ROS 2 packages, Python source code, and launch files;
- ▶ working Gazebo/PX4 inspection simulation;
- ▶ camera and object-detection integration.

Delivery Standard

A reviewer should be able to clone the repository, follow the documentation, reproduce the demonstration, and understand what was verified.

Evidence and Communication

- ▶ layer-level and end-to-end tests;
- ▶ logs, screenshots, and mission results;
- ▶ installation and execution instructions;
- ▶ final demonstration;
- ▶ technical report with results, limitations, and future work.

Knowledge Check

Can you explain the project before implementing it?

Mission and Architecture

1. What useful inspection outcome should the system produce?
2. Why is this a systems-engineering project?
3. Which components simulate physics, stabilize flight, and exchange information?

Verification and Boundaries

1. Why verify the camera stream before the detector?
2. Why is detection different from control?
3. Why does simulation success not prove real-flight readiness?

Expected Explanation

Describe the mission, command flow, evidence flow, role of each major technology, and the evidence required to reproduce the result.

Technology Ecosystem II

From programming logic to simulated drone intelligence

Python: The Logic

Implements readable robotics programs, data processing, mission utilities, tests, and AI integration.

ROS 2: The Communication Network

Connects software components using nodes, topics, messages, services, and parameters. ROS 2 is not an operating system.

Gazebo: The Virtual Test Environment

Simulates the drone, camera, objects, environment, and physics before any real-world deployment.

PX4: The Flight Controller

Estimates the drone state, stabilizes the vehicle, and follows flight references inside the simulation.

AI and YOLOv8: The Perception

Processes camera images and detects objects of interest. Detection results provide observations, not direct flight control.

Important Engineering Boundary

A successful simulation helps us develop and test the system, but it does not guarantee that the drone is ready for safe real-world flight.

Technology Ecosystem II

From programming logic to simulated drone intelligence

Python: The Logic

Implements readable robotics programs, data processing, mission utilities, tests, and AI integration.

ROS 2: The Communication Network

Connects software components using nodes, topics, messages, services, and parameters. ROS 2 is not an operating system.

Gazebo: The Virtual Test Environment

Simulates the drone, camera, objects, environment, and physics before any real-world deployment.

PX4: The Flight Controller

Estimates the drone state, stabilizes the vehicle, and follows flight references inside the simulation.

AI and YOLOv8: The Perception

Processes camera images and detects objects of interest. Detection results provide observations, not direct flight control.

Important Engineering Boundary

A successful simulation helps us develop and test the system, but it does not guarantee that the drone is ready for safe real-world flight.

How They Work Together

Build the environment before building the autonomous system

Build the environment before building the autonomous system

Next: Ubuntu and the Linux Engineering Environment

Ubuntu → Terminal and Bash → Git and GitHub → Python → ROS 2 → Gazebo and PX4 → AI integration

Guiding Question

What must be installed, organized, and verified so that every later component can run reliably inside the same engineering environment?

The Student Journey

Progress from guided reproduction to engineering ownership

1. **Orient:** understand the mission, architecture, repository, and expected evidence.
2. **Prepare:** configure Ubuntu, tools, dependencies, workspace, and project files.
3. **Learn the foundations:** practise Bash, Git, Python, ROS 2, Gazebo, PX4, and perception concepts in context.
4. **Integrate:** connect mission logic, simulated flight, camera data, and object detection.
5. **Verify:** test one layer at a time before testing the complete pipeline.
6. **Deliver:** publish reproducible code, configuration, evidence, documentation, and a technical report.